МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 4

з навчальної дисципліни

«Технології розробки корпоративних Web-додатків»

Виконала:

студентка 403-ТН

Кіріяк Ірина Вікторівна

Перевірив:

Мізік Андрій Олегович

Полтава 2026

Лабораторна робота №4
Реалізація підсистеми безпеки, автентифікації та авторизації

Передумова

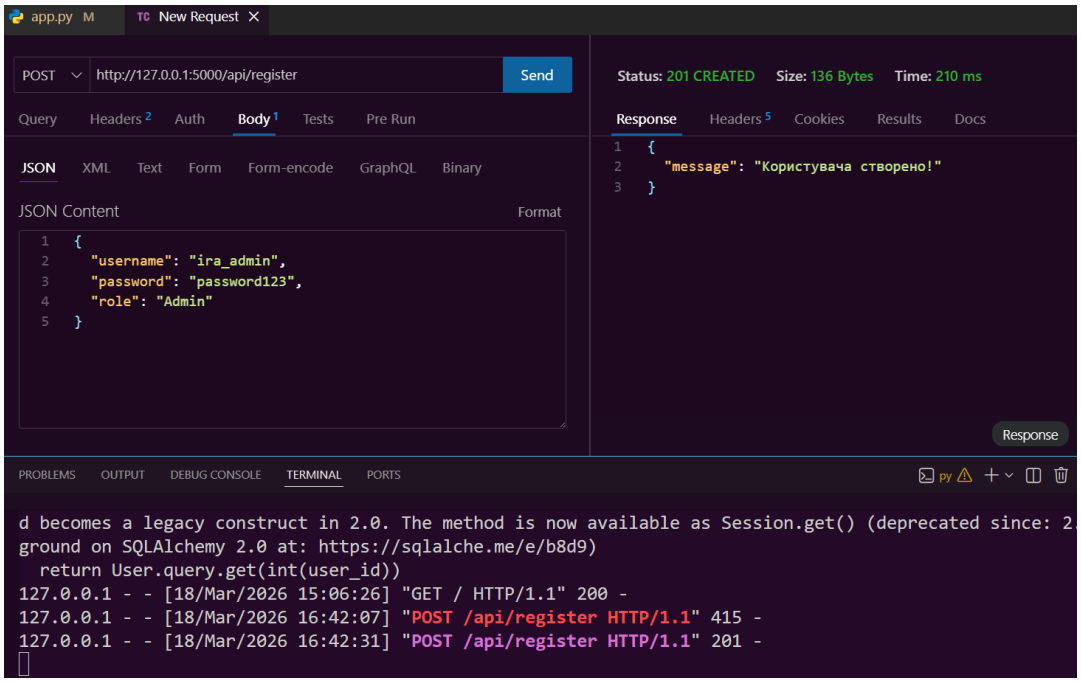
- Робота виконується в репозиторії *TRVD_2026_{Група}_{Прізвище}_Labs*.
- Використовується API, розроблений у Лабораторній роботі №3.
- Необхідно встановити бібліотеки для криптографії та роботи з токенами (наприклад, BCrypt, Argon2, System.IdentityModel.Tokens.Jwt, Passport.js, Spring Security тощо).

Мета роботи — захистити API від несанкціонованого доступу, реалізувати механізм реєстрації та входу користувачів, а також розмежувати права доступу на основі ролей.

1. Реалізація моделі та безпека

У проєкті реалізовано сутність User, яка містить поле role для розмежування прав доступу.

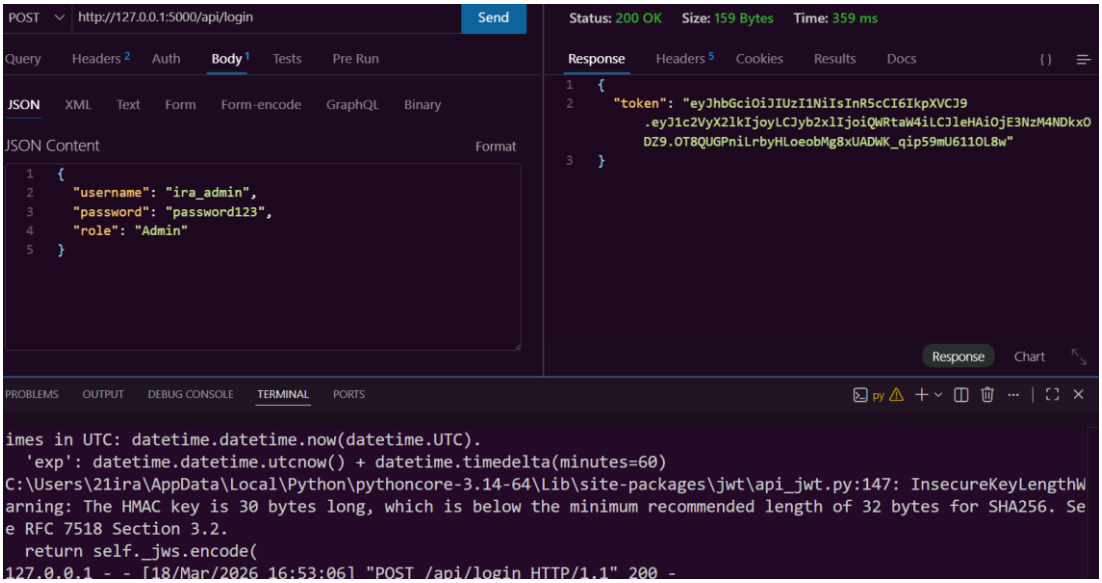
- Безпечне зберігання: Паролі хешуються за допомогою алгоритму PBKDF2 із використанням солі (salt) через функцію generate_password_hash.
- Реєстрація: Ендпоінт POST /api/register приймає дані, хешує пароль і зберігає нового користувача в базу SQLite.



2. Автентифікація та JWT

Для управління сесіями обрано механізм JWT (JSON Web Tokens), що є стандартом для REST API.

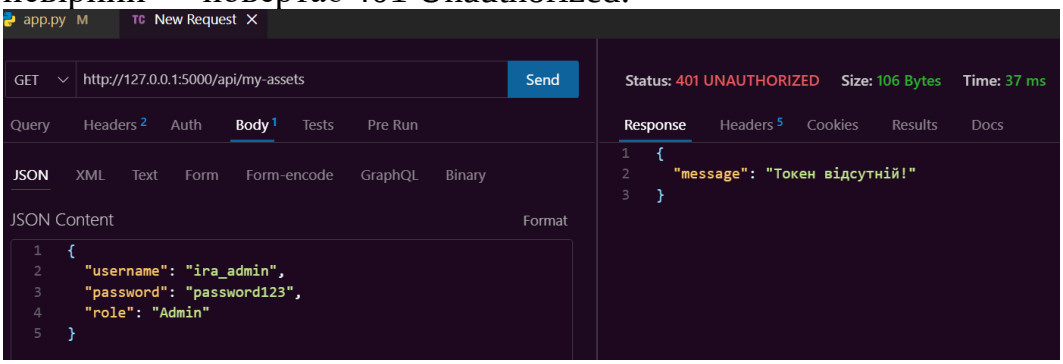
- Логін: Ендпоінт POST /api/login перевіряє відповідність введеного пароля хешу в БД за допомогою check_password_hash.
- Токен: При успішному вході генерується токен, що містить user_id та role. Термін дії обмежений 60 хвилинами для безпеки.



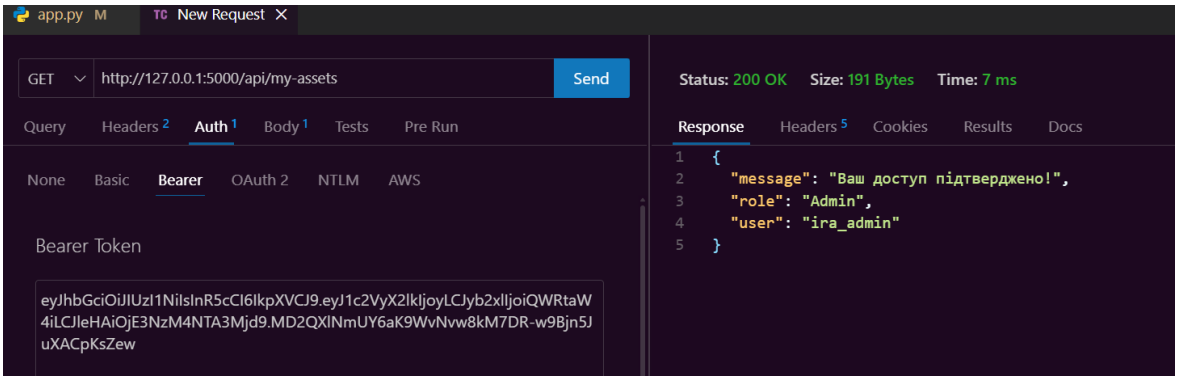
3. Авторизація (Middleware)

Для перевірки прав доступу реалізовано механізм перехоплення запитів через декоратори (Middleware):

1. @token_required: Перевіряє наявність заголовка Authorization: Bearer <token>, декодує його та перевіряє валідність. Якщо токена немає або він невірний — повертає 401 Unauthorized.



2. @admin_required: Перевіряє роль користувача, витягнуту з токена. Якщо роль не "Admin" — повертає 403 Forbidden.



Контрольні запитання

Різниця між автентифікацією та авторизацією

1. Автентифікація: Відповідь на питання "Хто ви?". Процес перевірки особи (наприклад, через логін/пароль).
2. Авторизація: Відповідь на питання "Що вам дозволено робити?". Перевірка прав доступу до конкретних ресурсів після того, як особу підтверджено.

Чому не можна зберігати паролі у відкритому вигляді?

При витоку бази даних зловмисники отримають доступ до всіх акаунтів. Сіль (Salt) — це випадковий рядок, що додається до пароля перед хешуванням. Вона захищає від атак за допомогою заздалегідь обчислених таблиць (Rainbow Tables), роблячи хеш кожного пароля унікальним, навіть якщо самі паролі однакові.

Структура JWT токена

1. Header: Містить тип токена та алгоритм шифрування (напр. HS256).
2. Payload: Корисні дані (Claims). Саме тут ми зберігаємо user_id та role.
3. Signature: Підпис, створений на основі заголовка, даних та секретного ключа сервера. Запобігає підробці токена.

Симетричне vs Асиметричне шифрування

1. Симетричне (HS256): Використовується один секретний ключ і для підпису, і для перевірки. Швидше, підходить для одного сервера.
2. Асиметричне (RS256): Використовується пара ключів (приватний для підпису, публічний для перевірки). Дозволяє стороннім сервісам перевіряти токен, не знаючи секретного ключа.

Атаки Brute Force та Rainbow Table

1. Brute Force: Перебір усіх можливих комбінацій паролів. Захист: обмеження кількості спроб входу, складні паролі.
2. Rainbow Table: Використання гігантських баз готових хешів. Захист: використання солі (Salt).

Принцип роботи RBAC

Role-Based Access Control — це модель, де права доступу групуються в "ролі" (наприклад, Admin, Editor, User). Користувачеві призначається одна або кілька ролей, і він отримує всі права, пов'язані з ними.